



Enterprise Trading Platform

Applied Capstone Project – India Cohort

Specification & Build Guide

A single platform, built progressively across the programme, from data layer to live cloud deployment.

Contents

PART A Overview & Approach

- 1** Executive Summary
- 2** Project Overview
- 3** Curriculum-Aligned Development Approach
- 4** Technology Stack
- 5** Architecture Overview

PART B Build Components

- 6** Component 1: Trade Database (Postgres)
- 7** Component 2: Analytics & ETL (Python)
- 8** Component 3: Core Business Logic (Java & OOAD)
- 9** Component 4: Trade REST API (Spring Boot)
- 10** Component 5: Event-Driven Backbone (Kafka & ETL)
- 11** Component 6: Authentication Service (NestJS / JWT)
- 12** Component 7: User Interface (Angular)
- 13** Component 8: Extension Microservice (Team's Choice)
- 14** Component 9: Cloud Deployment (AWS)

PART C Technical Specification

- 15** Functional Requirements
- 16** Non-Functional Requirements
- 17** Business Rules
- 18** Error Handling
- 19** Security Implementation

PART D Delivery & Assessment

- 20** Effort & Delivery Estimation
- 21** Deliverables
- 22** Evaluation Criteria

PART A

Overview & Approach

1. Executive Summary

1.1 Project Snapshot

Attribute	Details
Project Name	Enterprise Trading Platform
Project Type	Enterprise Financial Services Application (Microservices)
Development Approach	Curriculum-Aligned Progressive Build
Delivery Model	Team-based build across project sessions and a dedicated applied-project week
Team Size	4-5 participants per team
Build Sequence	Postgres → Python Analytics → Java / OOAD → Spring Boot → Kafka → NestJS Auth → Angular → Extension → AWS
AI Tooling	GitHub Copilot used throughout: code, tests, architecture and review

1.2 Objective Statement

Build a **production-grade enterprise trading platform** progressively, with each curriculum sprint contributing specific components to a single, integrated application.

Teams receive the written project specification, API and event contracts, representative fixtures, and non-runnable templates. No scripts, Docker Compose file, authentication stub, assessment harness, application boilerplate, or runnable implementation is supplied. Teams design and implement every assessed database and application artifact, then integrate and deploy the completed platform.

2. Project Overview

2.1 Problem Statement

Financial services firms run on systems that move trade and market data reliably and securely across many services, while maintaining strong identity controls, full auditability and operational resilience. The platform models this end-to-end: capturing trades against accounts, moving data in real time and in batch, exposing secure APIs, and presenting an authenticated user interface backed by analytics.

2.2 Solution Approach

Build the application **progressively** as each sprint is completed:

Sprint Focus	Project Component	Outcome
Data modelling (Postgres)	Trade database schema	Version-controlled data layer
Analytics & ETL (Python)	Dashboard + simple ETL pipeline	First data movement & insights
Java & OOAD	Core trading business logic	Clean, tested business engine
Spring Boot	Trade REST API	Functional backend service
Kafka + ETL	Event-driven data backbone	Real-time + batch data flow
NestJS / JWT	Authentication service	Security & identity layer
Angular	User interface	Complete full-stack application
Extension microservice	Team-selected capability	Distinctive team feature
AWS	Cloud deployment	Live, deployed system

3. Curriculum-Aligned Development Approach

3.1 Build Progression Overview

COMPONENT 1: TRADE DATABASE (Postgres) · Sprint 3

- Design a normalised relational schema for accounts, instruments, orders and positions
- Add constraints, indexes and historical trade data structures
- Load representative seed data; produce ER diagram

SHOWCASE Production-standard schema, each team member can explain the data model

COMPONENT 2: ANALYTICS & ETL (Python) · Sprint 4

- Exploratory data analysis on a realistic trading dataset
- Dashboard surfacing at least three business insights
- Data ingestion structured as a simple, testable ETL pipeline

SHOWCASE Python dashboard with insights, ingestion framed as an ETL pipeline

COMPONENT 3: CORE BUSINESS LOGIC (Java & OOAD) · Sprint 5

- Domain entities (Account, Instrument, Order, Position)
- Enums (AccountStatus, OrderSide, OrderStatus)
- DTOs with validation annotations; custom exception hierarchy
- Order placement / settlement logic (buy / sell rules)
- Unit tests for domain logic (TDD, JUnit)

SHOWCASE Clean, SOLID, unit-tested business engine, ready to wrap in services

COMPONENT 4: TRADE REST API (Spring Boot) · Sprint 6

- Layered Spring Boot service (controller → service → repository)
- MyBatis persistence against the Sprint 3 Postgres schema

- REST endpoints to OpenAPI specification (place and cancel orders, order history) plus a provided basic Portfolio REST API (positions, balances, priced against live market data); DTO validation
- Global exception handling and structured logging; Dockerised

SHOWCASE Working REST APIs exposing the core logic, testable and containerised

COMPONENT 5: EVENT-DRIVEN BACKBONE (Kafka & ETL) · Sprint 7

- Kafka topics, producers and consumers moving trade events between services; a Trade Executor consumes trade events and executes asynchronously via the provided Broker Simulator API
- Batch ETL pipeline (extract → transform → load) with error handling and retries
- Data quality validation and dead-letter handling
- Quality gates (SonarQube) on the pipeline

SHOWCASE Real-time and batch data backbone; dashboard consumes from both sources

COMPONENT 6: AUTHENTICATION SERVICE (NestJS / JWT) · Sprint 8

- Modular, layered NestJS service in TypeScript
- Secure JWT issuance, validation and token lifecycle
- Password hashing, guards, decorators and interceptors
- Auth-specific OWASP controls; live OpenAPI docs

SHOWCASE Centralised auth service issuing and validating JWTs, protecting a backend route

COMPONENT 7: USER INTERFACE (Angular) · Sprint 9

- Reactive Angular app (standalone components, signals)
- Authenticated login flow with JWT interceptor and route guards
- Order ticket, dashboard, positions and order-history views
- Typed API clients from OpenAPI; unit + end-to-end tests

SHOWCASE Authenticated full-stack application with core journeys tested

COMPONENT 8: EXTENSION MICROSERVICE (Team's Choice) · Sprint 10

- Select and build one or more extensions: Portfolio & P&L, Trade Advice & Signals, Watchlists & Price Alerts, Customer Notifications, Customer Preferences, or Automated Strategy Execution
- Integrate end-to-end with the existing platform
- Apply secure-by-design and code-quality practices throughout

SHOWCASE A distinctive, team-selected capability integrated into the platform

COMPONENT 9: CLOUD DEPLOYMENT (AWS) · Sprint 11

- Deploy the Angular build to S3 with origin access controls
- Front the bucket with CloudFront; validate the live application
- Automate the build → upload → invalidate deployment cycle

SHOWCASE A live, deployed cloud home with an automated deployment pipeline

3.2 Component Dependencies

Component	Depends On	Provides To
Trade Database	None	All components (data layer)
Analytics & ETL	Trade Database	Insight & reporting views
Core Business Logic	Trade Database	Trade REST API (domain)
Trade REST API	Core Business Logic	Angular UI, Kafka, Auth
Event-Driven Backbone	Trade REST API	Analytics, downstream services
Auth Service	Trade REST API	Angular UI (secured routes)
Angular UI	Spring Boot + Auth	End users
Extension Microservice	All core components	Distinctive platform capability
Cloud Deployment	Angular UI	Live, accessible system

4. Technology Stack

4.1 Stack by Build Component

Component	Technologies
Version Control	Git, GitHub, Pull Requests, GitHub Actions
Trade Database	PostgreSQL, SQL (CTEs, window functions), ER modelling
Analytics & ETL	Python, pandas, matplotlib / seaborn / plotly, pytest
Core Business Logic	Java 21, JUnit, SOLID, TDD, UML
Trade REST API	Spring Boot 3.x, MyBatis, Bean Validation, OpenAPI, Docker
Event-Driven Backbone	Apache Kafka, Python ETL, SonarQube
Auth Service	Node.js, NestJS, TypeScript, JWT, bcrypt/argon2, Jest
User Interface	Angular 21+, TypeScript, RxJS/Signals, Playwright
Cloud Deployment	AWS (S3, CloudFront, IAM), AWS CLI, GitHub Actions
AI Assistance	GitHub Copilot (across every component)

4.2 Complete Technology Matrix

Category	Technology	Version	Component
Version Control	Git / GitHub	2.x	All
Language (Backend)	Java	21 LTS	Business Logic / API

Category	Technology	Version	Component
Framework	Spring Boot	3.x	Trade REST API
Persistence	MyBatis	3.x	Trade REST API
Database	PostgreSQL	16.x	Data layer
Messaging	Apache Kafka	3.x	Event backbone
Analytics	Python / pandas	3.12	Analytics & ETL
Data Warehouse	Snowflake	Cloud	Analytics
Auth Runtime	Node.js / NestJS	20 LTS / 10.x	Auth service
Auth Tokens	JWT	n/a	Auth service
Frontend	Angular	21+	User interface
Testing	JUnit / Jest / Playwright	n/a	All layers
Quality	SonarQube	n/a	Backbone / Excellence
Cloud	AWS (S3 + CloudFront)	n/a	Deployment
AI Tooling	GitHub Copilot	n/a	All components

5. Architecture Overview

The platform is a layered set of services. The Angular front end calls two Java / Spring Boot services: a Trade REST API that holds the core trading logic (place and cancel orders, order validation against rules and risk, order history) and a Portfolio REST API that tracks portfolio, positions and balances priced against live market data. Teams design and implement both services, which persist to Postgres via MyBatis. A NestJS auth service issues and validates JWTs to secure those calls, with OAuth2-style login that is MFA-ready and role-based access control. Trade events flow over a Kafka event bus to downstream consumers and a batch ETL pipeline feeding analytics. A Trade Executor consumes trade events from Kafka and interacts asynchronously with a provided Broker Simulator API to execute trades. Two external services are provided: a Live Pricing API supplies market data and a Broker Simulator API simulates asynchronous trade execution. Security is enforced at every boundary (TLS 1.2+ in transit, signed JWT and RBAC, input validation and risk checks, parameterised SQL with least-privilege roles, and Kafka TLS, SASL and ACLs with a full audit trail). Teams add one or more optional microservices to extend the platform.

5.1 Layered Architecture

Layer	Component	Responsibility
Presentation	Angular UI (Sprint 9)	Authenticated front end: order ticket, dashboard, blotter

Layer	Component	Responsibility
Security	NestJS Auth Service (Sprint 8)	JWT issuance, validation and access control
API	Spring Boot Trade & Portfolio REST APIs (Sprint 6)	Order endpoints (place, cancel, history), portfolio and positions, validation, orchestration
Domain	Core Business Logic (Sprint 5)	Trading rules, order lifecycle, positions
Messaging	Kafka Event Bus (Sprint 7)	Trade events between services; producers/consumers
Execution	Trade Executor (Sprint 7)	Consumes trade events; executes asynchronously via the provided Broker Simulator API
Data	PostgreSQL (Sprint 3)	Accounts, instruments, orders, positions
Analytics	Python ETL + Snowflake (Sprint 4 & 7)	Batch ETL, dashboards, business intelligence
External	Live Pricing API & Broker Simulator API Contracts	Market prices for instruments; asynchronous trade execution
Version Control	Git & GitHub	Branching, PR review, CI

5.2 Extension Microservices

Extension	Purpose
Portfolio & P&L service	Holdings and profit & loss across positions
Trade advice & signals	Research-driven BUY / SELL / HOLD guidance with the reasoning behind each
Automated strategy execution	Rules-based strategies (moving-average crossovers, Bollinger-band triggers) acting on the real-time price stream
Customer notifications	Proactive, real-time alerts (trade confirmations, price moves, statements, corporate actions) via the customer's preferred channel
Watchlists & price alerts	Watchlists with price or volatility thresholds that trigger real-time alerts
Customer preferences & personalisation	Persisted settings and defaults (accounts, selections, alert channels) applied at next login

Teams select one or more extensions to make their build distinctive. **GitHub Copilot** is used for code generation, architecture decisions, test writing, code review and debugging across every component.

PART B

Build Components

6. Component 1: Trade Database (Postgres)

6.1 Component Overview

Attribute	Details
Sprint	Sprint 3: Data Systems & Data Modelling
Project Component	Trade database schema & data layer
Technology	PostgreSQL, SQL, ER modelling
Prerequisites	Version control and environment set up

6.2 Tasks & Deliverables

#	Task	Description	Focus
1	Conceptual model	Entities, relationships, ER diagram	Modelling
2	Normalised schema	1NF–3NF relational design	Design
3	Tables & constraints	Keys, foreign keys, check constraints	DDL
4	Indexes	Indexing for query performance	Performance
5	Historical structures	Support for historical trade data	Extension
6	Seed data	Representative accounts, instruments, trades	Data load

6.3 Component Showcase

Showcase Item	Description
Schema	Normalised, production-standard schema in Postgres
ER Diagram	Clear entity-relationship documentation
Seed Data	Representative data loaded and queryable
Explanation	Each team member can explain the data model
Git Commit	Feature branch merged to develop

7. Component 2: Analytics & ETL (Python)

7.1 Component Overview

Attribute	Details
Sprint	Sprint 4: Financial Services & Data Analytics
Project Component	Analytics dashboard & first ETL pipeline
Technology	Python, pandas, matplotlib / seaborn / plotly, pytest
Prerequisites	Trade database (Component 1)

7.2 Tasks & Deliverables

#	Task	Description	Focus
1	Dataset	Add a realistic sample trading dataset	Data
2	EDA	Explore patterns, anomalies, segments	Analysis
3	Cleaning	Handle missing values, types, outliers	Preparation
4	Insights	Surface at least three business insights	Reporting
5	Visualisation	Clear charts for a non-technical audience	Communication
6	Simple ETL	Structure ingestion as extract-transform-load	Pipeline
7	Pipeline tests	pytest on transforms and outputs	Quality

7.3 Example Analytics

Analytic	Purpose
Trade Volume	Count and value of trades by date
Most Active Accounts	Accounts with highest trading activity
Fill Rate	Percentage of orders filled vs rejected
Exposure by Instrument	Aggregate positions by instrument
Average Trade Size	Mean trade value over a period

7.4 Component Showcase

Showcase Item	Description
Dashboard	Python dashboard surfacing 3+ insights
ETL Pipeline	Ingestion structured as a testable ETL flow
Tests	pytest validating transforms and outputs
Git Commit	Feature branch merged to develop

8. Component 3: Core Business Logic (Java & OOAD)

8.1 Component Overview

Attribute	Details
Sprint	Sprint 5: Software Engineering Essentials, Java & OOAD
Project Component	Core trading business logic
Technology	Java 21, JUnit, SOLID, TDD, UML
Prerequisites	Trade database (Component 1)

8.2 Tasks & Deliverables

#	Task	Description	Focus
1	Domain modelling	UML class & sequence diagrams	OOAD
2	Entities	Account, Instrument, Order, Position	Domain
3	Enums	AccountStatus, OrderSide, OrderStatus	Domain
4	DTOs	Request/response objects with validation	Contracts
5	Exceptions	Custom exception hierarchy	Robustness
6	Order logic	Buy/sell rules, position updates	Business
7	Unit tests	TDD red-green-refactor with JUnit	Quality

8.3 Component Showcase

Showcase Item	Description
UML Artefacts	Class and sequence diagrams for the trading domain
Domain Package	Entity classes ready for persistence
DTO Package	Request/response objects with validation
Exception Package	Custom exception hierarchy
Unit Tests	All tests passing (green)
Git Commit	Feature branch merged to develop

9. Component 4: Trade REST API (Spring Boot)

9.1 Component Overview

Attribute	Details
Sprint	Sprint 6: Software Architecture & Enterprise Java
Project Component	REST API, services, persistence
Technology	Spring Boot 3.x, MyBatis, Bean Validation, OpenAPI, Docker
Prerequisites	Core business logic (Component 3)

9.2 Tasks & Deliverables

#	Task	Description	Focus
1	Project setup	Spring Boot project & dependencies	Setup
2	API contract	OpenAPI spec before implementation	Design
3	Layered design	Controller → service → repository	Architecture
4	MyBatis mappers	Persistence against the Sprint 3 schema	Data access
5	OrderService	Wrap core logic with transactions	Business
6	Controllers	Order (place, cancel, history) and portfolio / account REST endpoints	API
7	Validation & DTOs	Bean Validation, request/response shaping	Contracts
8	Exception handler	@ControllerAdvice implementation	Robustness
9	Containerisation	Dockerise the service	Delivery

9.3 Component Showcase

Showcase Item	Description
Running Service	Spring Boot app starts and serves requests
API Demo	Endpoints tested and working
Persistence	Reads/writes against Postgres via MyBatis
Container	Service runs as a Docker image
Git Commit	Feature branch merged to develop

10. Component 5: Event-Driven Backbone (Kafka & ETL)

10.1 Component Overview

Attribute	Details
Sprint	Sprint 7: Enterprise Data & Engineering Excellence
Project Component	Real-time eventing + batch ETL
Technology	Apache Kafka, Python ETL, SonarQube
Prerequisites	Trade REST API (Component 4)

10.2 Tasks & Deliverables

#	Task	Description	Focus
1	Topic design	Topics, partitions for trade events	Design
2	Producers	Emit trade events from services	Real-time
3	Consumers	Consume and process trade events	Real-time
4	Batch ETL	Extract from Postgres, transform, load	Pipeline
5	Resilience	Error handling, retries, idempotency	Robustness
6	Data quality	Validation, schema checks, dead-letter queue	Quality
7	Quality gates	SonarQube analysis on the pipeline	Excellence

10.3 Component Showcase

Showcase Item	Description
Topics & Flows	Topics, producers and consumers implemented and verified
Batch ETL	Trade data extracted, transformed and loaded
Dashboard	Updated to consume from real-time and batch sources
Quality	SonarQube quality gates applied
Git Commit	Feature branch merged to develop

11. Component 6: Authentication Service (NestJS / JWT)

11.1 Component Overview

Attribute	Details
Sprint	Sprint 8: Node.js / NestJS & Authentication
Project Component	Centralised authentication service
Technology	Node.js, NestJS, TypeScript, JWT, bcrypt/argon2, Jest

Attribute	Details
Prerequisites	Trade REST API (Component 4)

11.2 Tasks & Deliverables

#	Task	Description	Focus
1	Project setup	Scaffold a layered NestJS service	Setup
2	API contract	OpenAPI-first auth endpoints	Design
3	Endpoints	login, register, refresh	API
4	Password hashing	bcrypt / argon2 with salts	Security
5	JWT issuance	Login flow returning a signed token	Auth
6	JWT validation	Verify tokens on protected routes	Auth
7	Guards & decorators	Protect routes, extract user from token	Access control
8	Unit tests	Jest tests, including guards	Quality
9	Integration	Protect a Spring Boot route end-to-end	Integration

11.4 Component Showcase

Showcase Item	Description
Auth Service	Issues and validates JWTs
Protected Route	At least one Spring Boot route secured by the service
OpenAPI Docs	Live, auto-generated documentation
Security Review	Auth flow reviewed against OWASP auth items
Git Commit	Feature branch merged to develop

12. Component 7: User Interface (Angular)

12.1 Component Overview

Attribute	Details
Sprint	Sprint 9: UI Development with Angular
Project Component	Single-page application (UI)
Technology	Angular 21+, TypeScript, RxJS / Signals, Playwright
Prerequisites	Trade REST API and Auth service

12.2 Tasks & Deliverables

#	Task	Description	Focus
1	Project setup	Standalone components, routing	Setup
2	Auth + interceptor	Login, token storage, JWT interceptor	Security
3	Route guards	Protect authenticated routes	Security
4	API clients	Typed clients from OpenAPI specs	Integration
5	Dashboard	Balance, positions, market overview	Feature
6	Order ticket	Place-order form with validation	Feature
7	Blotter / history	Order history with status badges	Feature
8	Tests	Unit + Playwright end-to-end tests	Quality

12.3 Component Showcase

Showcase Item	Description
Running App	ng serve working
Login Flow	Authentication working end-to-end
Dashboard	Balance and positions from the API
Order Flow	End-to-end order placement working
Tests	Unit and Playwright tests covering core journeys
Git Commit	Feature branch merged to develop

13. Component 8: Extension Microservice (Team's Choice)

13.1 Component Overview

Attribute	Details
Sprint	Sprint 10: Applied Project Week
Project Component	Team-selected extension microservice
Technology	Team's choice, integrated with the platform
Prerequisites	All core components integrated

13.2 Extensions

Extension	What the team builds
Portfolio & P&L service	Consolidated holdings, cost basis and realised / unrealised P&L across accounts; exposed via REST API. Builds on: Trade Database (S3) and trade events (S7).
Trade advice & signals	Research-driven BUY / SELL / HOLD signals against holdings and watchlist, with the reasoning behind each. Builds on: market & price data (S4) and the Portfolio service; analytics layer.
Automated strategy execution	Rules-based strategies (moving-average crossovers, Bollinger-band triggers) that watch the real-time price stream and act when conditions are met. Builds on: real-time price stream (S7) and Trade REST API (S6).
Customer notifications	Proactive, real-time alerts (trade confirmations, price moves, statements / eBill, corporate actions) via each customer's preferred channel. Builds on: Kafka event stream (S7) and Customer Preferences.
Watchlists & price alerts	Watchlists with price or volatility thresholds that trigger real-time alerts. Builds on: real-time price stream (S7, Kafka) and the Notifications service.
Customer preferences & personalisation	Persisted settings and defaults (default accounts, selections, alert channels), set explicitly or learned through interaction and applied at next login. Builds on: Auth & profile service (S8) and persistence.

13.3 Tasks & Deliverables

#	Task	Description	Focus
1	Scope confirmation	Confirm extension and integration plan	Planning
2	Build	Develop the chosen microservice	Delivery
3	Integration	Connect to Spring Boot APIs and auth	Integration
4	Security review	Apply OWASP principles to the new service	Security
5	Quality pass	SonarQube, test coverage, refactoring	Excellence
6	Decision log	Capture the rationale for the showcase	Documentation

13.4 Component Showcase

Showcase Item	Description
Working Extension	Chosen service integrated end-to-end
Architecture Log	Decisions captured for the showcase
Quality	Code quality and security checks passed
Git Commit	Feature branch merged to develop

14. Component 9: Cloud Deployment (AWS)

14.1 Component Overview

Attribute	Details
Sprint	Sprint 11: Cloud, Deployment & Final Showcase
Project Component	Live cloud deployment & automation
Technology	AWS (S3 + CloudFront, IAM), AWS CLI, GitHub Actions
Prerequisites	Full-stack application complete

14.2 Tasks & Deliverables

#	Task	Description	Focus
1	Account & IAM	Account setup, IAM basics, region	Setup
2	AWS CLI	Authenticate and verify access	Tooling
3	S3 hosting	Deploy the Angular build to S3	Deploy
4	Origin access	Keep the bucket private, served via CDN	Security
5	CloudFront	Stand up CDN in front of the bucket	Deploy
6	Validation	Verify authenticated flows in production	Verify
7	Automation	Script build → upload → invalidate	Pipeline

14.3 Component Showcase

Showcase Item	Description
Live Deployment	Application reachable via CloudFront
Origin Controls	Private bucket, served only via the CDN
Automation	Repeatable deployment script / Action
Final Showcase	Live demo to leaders, instructors and peers

PART C

Technical Specification

15. Functional Requirements

15.1 Core Features

ID	Feature	Component	Description
FR-01	Order Placement & Cancellation	Spring Boot	Place buy / sell orders against an account and cancel working orders
FR-02	Account Validation	Spring Boot	Verify account existence and status
FR-03	Instrument Validation	Spring Boot	Verify the instrument is tradable
FR-04	Funds / Holdings Check	Spring Boot	Ensure sufficient cash (buy) or holdings (sell)
FR-05	Order Logging	Spring Boot	Record all orders (audit trail)
FR-06	Idempotency	Spring Boot	Prevent duplicate orders
FR-07	Balance & Positions	Spring Boot + Angular	View cash balance and holdings, priced against live market data
FR-08	Order History	Spring Boot + Angular	View past orders
FR-09	Authentication	NestJS + Angular	Secure login with JWT
FR-10	Eventing	Kafka	Publish and consume trade events
FR-11	Analytics	Python / Snowflake	Business insights and reporting
FR-12	Trade Execution	Kafka / Trade Executor	Execute trades asynchronously via the provided Broker Simulator API contracts

16. Non-Functional Requirements

ID	Requirement	Component	Implementation
NFR-01	Transactional integrity	Spring Boot	@Transactional order processing
NFR-02	Concurrency control	Spring Boot	Optimistic locking (version)
NFR-03	Security	NestJS / Spring	JWT auth + Bean Validation

ID	Requirement	Component	Implementation
NFR-04	Auditability	Spring Boot / Kafka	Order logging and trade events
NFR-05	Resilience	Kafka / ETL	Retries, idempotency, dead-letter queue
NFR-06	Code quality	All	SonarQube quality gates, test coverage
NFR-07	Version Control	Git	Branching and pull-request review

17. Business Rules

#	Rule	Error Code
1	Account must exist	ACC-404
2	Account must be ACTIVE	ACC-403
3	Instrument must exist and be tradable	INS-404
4	Quantity must be greater than 0	VAL-422
5	Price must be greater than 0	VAL-422
6	BUY: cash balance $\geq$ quantity $\times$ price	ORD-400
7	SELL: held quantity $\geq$ order quantity	ORD-409
8	Idempotency key must be unique	ORD-409
9	Update cash and position atomically	n/a
10	Log every order	n/a

18. Error Handling

18.1 Error Catalog

Code	Message	HTTP
ACC-404	Account not found	404
ACC-403	Account not active	403
INS-404	Instrument not found	404
ORD-400	Insufficient funds	400
ORD-409	Insufficient holdings / duplicate order	409
VAL-422	Invalid input	422

Code	Message	HTTP
AUTH-401	Unauthorised / invalid token	401

19. Security Implementation

Feature	Implementation
Authentication	JWT issued and validated by the NestJS auth service
Protected Routes	All /api/** endpoints require a valid token
Input Validation	Bean Validation (Java) and class-validator (NestJS)
Password Storage	bcrypt / argon2 hashing with salts
Secure Coding	OWASP Top 10 applied at every layer (Secure Code Warrior)
Pipeline Security	DevSecOps checks: SAST, dependency and secret scanning

PART D

Delivery & Assessment

20. Effort & Delivery Estimation

20.1 Summary by Build Component

Indicative project time only; this sits alongside the instructor-led learning delivered in each sprint. The applied project runs as project sessions across Sprints 3–9, a dedicated applied-project week (Sprint 10) and deployment with the final showcase (Sprint 11).

Component	Sprint	Project Session	Indicative
Trade Database	3	Project session	~4 hrs
Analytics & ETL	4	Project session	~4 hrs
Core Business Logic	5	Project session	~4 hrs
Trade REST API	6	Project session	~4 hrs
Event-Driven Backbone	7	Project session	~4 hrs
Auth Service	8	Project session	~4 hrs
Angular UI	9	Project session	~4 hrs
Extension + Integration	10	Applied-project week	~30 hrs
Cloud Deployment & Showcase	11	Deployment + showcase	~12 hrs
Total (indicative)			~78 hrs

21. Deliverables

21.1 Deliverables by Component

Component	Deliverables	Showcase
Trade Database	Schema, ER diagram, seed data	Data-model walkthrough
Analytics & ETL	Dashboard, ETL pipeline, tests	Insights demo
Business Logic	Domain classes, DTOs, exceptions, tests	Tests passing
Trade REST API	Endpoints, services, persistence, container	API demo
Event Backbone	Kafka topics, ETL pipeline	Event + batch flow
Auth Service	JWT service, OpenAPI docs	Secured route demo
Angular UI	SPA with core screens, tests	Full-stack demo

Component	Deliverables	Showcase
Extension	Team-selected microservice	Integrated capability
Cloud Deployment	Live deployment, automation script	Live system demo

22.2 Final Deliverables

#	Deliverable	Description
1	GitHub Repository	Complete source code with branch history
2	Backend Services	Spring Boot API, Kafka components, NestJS auth
3	Frontend Application	Working Angular app
4	Database Scripts	Postgres DDL + seed data
5	Analytics & ETL	Python pipeline and dashboard
6	Tests	Passing unit and end-to-end suites
7	Live Deployment	Application deployed to AWS
8	Documentation	README with setup and run guide

23. Evaluation Criteria

23.1 Component-wise Evaluation

Component	Weight	Criteria
Data & Analytics	15%	Schema quality, insights, ETL
Business Logic (Java)	20%	Domain model, OOAD, SOLID, tests
Trade REST API	20%	API design, services, persistence
Eventing & Excellence	10%	Kafka, quality gates, refactoring
Auth & Security	15%	JWT, OWASP, secure coding
User Interface	15%	Components, integration, tests
Deployment & Showcase	5%	Live deployment, presentation
Total	100%	

23.2 Cross-Cutting Assessment

Across all components, participants are assessed on responsible GitHub Copilot use (with mandatory code walkthroughs and peer reviews confirming genuine understanding), engineering excellence (clean

code, meaningful test coverage, quality gates), secure-by-design practices, and clear communication of design decisions to technical and non-technical audiences.